# Intent Compression Architecture

A Clarification-First Control Layer for Reliable LLM Systems

Engineering design proposal

| | |
|---|---|
| **Author** | Paul Maddison |
| **Short name** | ICA |
| **Canonical artifacts** | README.md, architecture diagram, proposal DOCX/PDF, evaluation scaffold |

Executive summary. ICA is a pre-generation control layer that decides whether clarification is worth the cost before the model commits to an answer. It treats ambiguity as uncertainty over latent user intent, estimates expected utility over possible clarification replies, and routes the request to direct answer, clarifier, premise-check, or refusal/redirect as appropriate.

## 1. Architecture overview

A common failure mode in LLM systems is unresolved ambiguity. Users often ask questions that admit multiple plausible interpretations, while the system is optimized to answer immediately. The result is a broad first answer, a user correction, and an unnecessary second pass.

ICA inserts a control layer between user input and final generation. The layer estimates likely intent hypotheses, scores ambiguity and risk, and decides whether clarification improves the expected outcome enough to justify the extra turn.

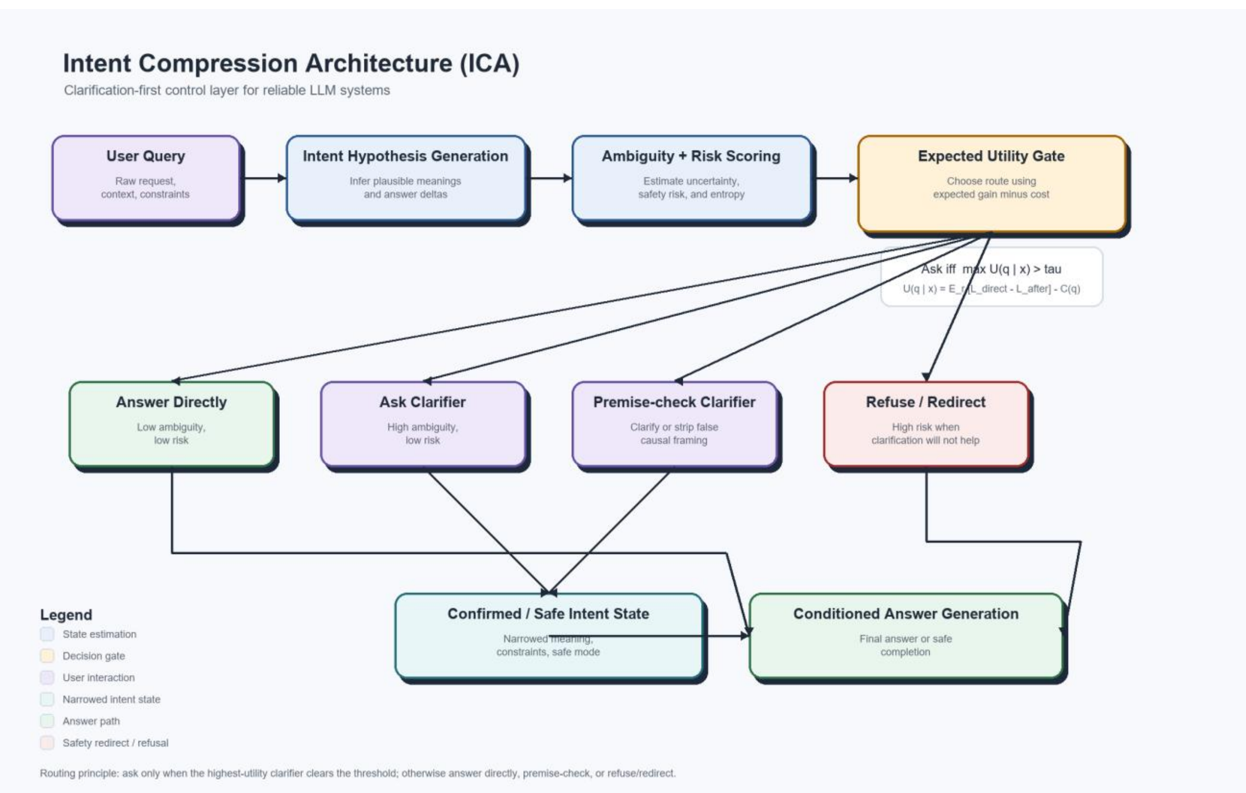

*Figure 1. ICA control flow with expected-utility routing and embedded legend.*

## 2. Core decision policy

Minimal rule: ask a clarifying question only when expected improvement in answer quality or safety exceeds the cost of another turn.

```
U(q | x) = E_r [L_direct(x) - L_after(x, q, r)] - C(q)

Ask iff max_q U(q | x) > tau_domain
```

This formulation makes the unknown user reply explicit by taking an expectation over possible clarification responses. The domain threshold allows the same architecture to be tuned differently for coding, medical, legal, finance, or customer-support use cases.

## 3. Compression interpretation

ICA treats ambiguity as entropy over possible user intents. Before clarification, the system reasons over $H(I \mid x)$. After clarification q and user reply r, the system reasons over $H(I \mid x, q, r)$. The expected value of clarification is the reduction in intent entropy, tempered by interaction cost.

This is why the term intent compression is mathematically defensible: the controller narrows the intent distribution before generation rather than letting the answering model hedge across multiple meanings.

## 4. Routing policy

| Ambiguity | Risk | Default action | Operational note |
|---|---|---|---|
| Low | Low | Answer directly | No clarification needed. |
| High | Low | Ask one high-value clarifier | Clarify only if the best question clears the threshold. |
| Low | High | Direct safe completion, premise-check, or refuse/redirect | Do not ask the user to reconfirm harmful intent if the safe response would not change. |
| High | High | Strict clarification, constrained answer, or refuse/redirect | High uncertainty plus high downside requires tighter control. |

A practical implication is that clarification is not always the right response to risk. Sometimes the cleanest and safest route is a direct refusal with a helpful redirect.

# 5. Implementation contract

The control layer should emit a structured decision object rather than a free-form paragraph. This proposal includes a JSON schema and reference example in spec/clarifier_output.schema.json and spec/clarifier_output.example.json.

The repository also includes a lightweight schema validation check so the example can be tested against the contract rather than presented as a static mockup.

```
{
  "ambiguity_score": 0.74,
  "risk_score": 0.22,
  "decision": "ask_clarifier",
  "clarifying_question": "When you say propaganda, do you mean persuasive political advocacy,
coordinated deceptive messaging, or something else?",
  "answer_constraints": ["avoid loaded framing", "define terms before conclusion"]
}
```

# 6. Hidden baseline cost

A short first answer is not necessarily an efficient answer. In ambiguous prompts, a baseline system can choose the wrong interpretation, give a broad or misleading answer, and force the user into a correction funnel before the real issue is finally exposed.

This is not only a retry problem. It is a wrong-funnel problem. The user either spends turns arguing the model into discovering the ambiguous term, or leaves with a partially wrong answer without ever learning what caused the mismatch.

For that reason, ICA should be evaluated on tokens to resolved intent, definition-discovery turn, correction-funnel depth, and user correction burden, not just on the cost of the first assistant response.

# 7. Evaluation package

The repository now includes a benchmark prompt set, an evaluation protocol, a sample reporting format, and a first-pass pilot benchmark. The evaluation compares not only direct one-shot answers, but also direct answers followed by the repair funnel when the first answer misses the intended meaning. The next credibility jump is a multi-rater or live-user benchmark.

The published pilot is designed to test ambiguous-prompt handling, not to estimate production-wide clarification frequency.

| Artifact | Purpose |
| --- | --- |
| examples/ambiguous_prompts.csv | Starter prompt set covering low-risk ambiguity, high-risk ambiguity, and premise-risk cases. |
| eval/README.md | Protocol, scoring rubric, success criteria, and counter-metrics. |
| eval/sample_results.md | Template for benchmark reporting without fabricating results. |

| | |
|---|---|
| eval/pilot_results.md | Human-readable first-pass pilot benchmark report. |
| eval/pilot_results.csv | Machine-readable pilot results table for analysis and reuse. |

Key primary metrics include first assistant-message tokens, total tokens to resolved intent, definition-discovery turn, retry count, correctness, clarity, and premise handling. Key counter-metrics include over-clarification rate, false direct-answer rate, silent-failure proxy, false refusal rate, and clarification bias.

## 8. Strategic implication

At large scale, ICA is not only a reliability pattern. It becomes a clarification data flywheel. Each ambiguous query, clarifier, user reply, and final outcome produces a structured intent-resolution trace that is more valuable than an ordinary chat log because it captures what the user actually meant.

Public OpenAI disclosures in 2026 describe ChatGPT as having more than 900 million weekly active users and more than 50 million consumer subscribers. At that scale, even rare ambiguity classes become common in absolute terms. A provider with that distribution can improve ambiguity detection, ask-vs-answer thresholds, neutral clarifier wording, risk handling, and future base-model behavior using real-world traces rather than only synthetic or expert-labeled data.

The architecture is copyable. The live, high-volume intent-resolution feedback loop is much harder to copy. That is why ICA can be understood not only as a UX improvement, but as a route by which market share becomes model-quality advantage.

## 9. Deployment guidance

ICA is most useful when deployed as a policy layer around an existing model stack. The orchestration logic should be explicit and testable, while uncertainty is isolated to model outputs and external-system calls such as retrieval, APIs, or mutable external state.

The controller should log ambiguity score, risk score, candidate clarifiers, expected utility estimates, final route, and user reply when applicable. Those traces are the data needed to improve the control layer over time.

## 10. Conclusion

ICA is a credible architecture pattern because it defines a control-layer problem that engineers can implement: infer intent hypotheses, quantify ambiguity, route by expected utility, narrow intent before generation when doing so is worth the cost, and build systems that are more precise, more reliable, easier to defend, and often cheaper to operate once wrong-funnel conversations are counted properly.

## Appendix A. Starter benchmark pack

The repository includes a lightweight evaluation package so the proposal can move from theory to early evidence without requiring a large research program.

| Artifact | Use |
| --- | --- |
| examples/ambiguous_prompts.csv | Seed prompt set covering coding, planning, legal, medical, finance, public reasoning, and safety-sensitive ambiguity. |
| eval/README.md | Evaluation procedure, rating rubric, success criteria, and counter-metrics. |
| eval/sample_results.md | Reporting format for summary metrics and per-prompt comparisons. |
| eval/pilot_results.md | Published first-pass pilot benchmark report. |
| eval/pilot_results.csv | Structured pilot dataset for re-analysis or extension. |

## Pilot signal snapshot

In the current 25-prompt pilot, direct first-pass answers required a repair funnel in 19 cases. On that same prompt set, ICA reduced mean definition-discovery turn from 2.6 to 1.0 and reduced mean total tokens to satisfactory resolution from 71.68 on the repaired baseline path to 62.08.

That is the narrower but stronger efficiency claim: early clarification is not always shorter than a one-shot answer, but it is often cheaper than discovering the same ambiguity after the system has already committed to the wrong semantic funnel.

In repaired-baseline scoring, final answer quality is equalized with ICA only when the baseline needed repair. The repaired utility score still penalizes extra repair tokens and retry burden so delayed clarification does not receive a free tie.

Reproduction path: install dependencies from requirements.txt, validate the schema/example pair, run eval/build_pilot_report.py, inspect the generated CSV and Markdown report, then regenerate the proposal artifacts if needed.

Recommended next move: extend the single-rater pilot into a multi-rater or API-instrumented benchmark, then update the controller threshold and routing rules based on observed over-clarification, correction-funnel depth, silent-failure risk, false direct-answer, and false-refusal rates.